Temporal Discretisation Shapes Optimisation Landscape in Spiking Neural Networks

Prakriti Parthasarathy[1], Thiago B. Burghi[1], and Timothy O'Leary[1]

[1]Department of Engineering, University of Cambridge, Cambridge, United Kingdom

1 Introduction

Spiking neural networks (SNNs) are widely regarded as the most biologically plausible paradigm for mimicking neural computation. Although they are often implemented on traditional von Neumann architectures, they also underpin neuromorphic computing hardware, which promise much greater energy efficiency compared to standard artificial neural networks (ANNs). Hence, it is of interest to computational neuroscientists and neuromorphic engineers to develop functional algorithms which achieve state of the art performance in SNNs.

Currently, a fundamental challenge in implementing SNNs is their trainability []. The binary, event-driven nature of spikes produces a highly discontinuous and mostly non-differentiable loss landscape, making it impossible to implement standard gradient descent methods to train SNNs. A widely adopted solution is surrogate gradient (SG) learning, in which discontinuous spiking functions are replaced with a differentiable approximation, allowing the network to be trained using backpropagation through time (BPTT) algorithms. This approach has enabled many studies to implement SNNs achieving performance comparable to ANNs on a range of tasks [].

Despite this, the theoretical foundations of surrogate gradient learning remain poorly understood []. In particular, little attention has been paid to the nature of temporal discretisation and its interaction with the network's gradients. Spiking dynamics are highly sensitive to both the discretisation scheme employed as well as the temporal resolution of the simulation. This sensitivity is a primary contributor to the low reproducibility of SNN results across implementations []. While prior work has attempted to reconcile differences across discretisation schemes [NIR], these efforts have largely focussed on single-cell or low-level validation, without examining its implications on network-level learning, generalisation, and training dynamics. Specifically, the effect of discretisation on the interaction between forward dynamics, surrogate gradient computation, and learning remains uncharacterised. In addition, the input parameters used in many SNN studies implementing SG learning are unrealistic and rely on a near-continuous rate-coded spiking input (Table 1). The cortex fires within a range of 5-50Hz when stimulated [], and neuromorphic hardware is designed to produce activity within 1-50Hz for energy efficient operation []. This results in an undesirable gap between the behaviour of the systems we wish to examine and the studies attempting to explain them. In this paper, we use single-neuron and network-level analyses to show that spiking patterns and synaptic drive are heavily dependent on the chosen discretisation scheme as well as the temporal resolution dt of the simulation. We study how discretisation induces dt dependent changes in both forward neural dynamics and backward gradient flow to degrade SG learning as $dt \to 0$ and as input rates become more biologically plausible. Lastly, we illustrate numerically and analytically how the loss interacts with SNN trainability and how we can redefine loss computation to recover performance in these regimes.

2 Results

2.1 Discretisation Alters Single-Neuron Responses to Poisson Spike Input

To motivate these issues, we begin by presenting the simplest possible case: a single postsynaptic neuron driven by N presynaptic spike trains. Fig 1A shows how different discretisation schemes (described in Appendix 1) affect the neuronal response of a postsynaptic neuron to identical frozen input Poisson spike trains. Despite receiving presynaptic events of equivalent input statistics, the resulting voltage trajectories, output spike timings, and overall postsynaptic neuronal dynamics can diverge across discretisation schemes. We then choose a fixed discretisation scheme we call 'Standard SNN', where the postsynap-

tic membrane potential V at timestep $k+1$ is given as:

$$V[k+1] = (\beta V[k] + WI[k+1])(1 - S[k+1]) \quad (1)$$

where $\beta = e^{\frac{-dt}{\tau_m}}$ is the membrane decay constant for a given simulation time-step dt and membrane time constant τ_m, the input I is scaled by a synaptic weight W, and S is the reset defined as:

$$S[k+1] = \begin{cases} 1, & \text{if } V[k+1] \geq \theta \\ 0, & \text{otherwise} \end{cases}$$

where θ is the threshold. This corresponds to a discrete-time LIF neuron with exponential membrane decay, equivalent to the exact solution of the continuous-time LIF under impulse-invariant input with instantaneous current-based synapses. This formulation is widely adopted in SG trained SNNs [] and mimics the zero synaptic time constant limit of LIF models with exponential synapses (Appendix 1E), used in snnTorch, Nengo, Rockpool, and SpiNNaker2 (exponential mode) [], and closely approximated by Euler formulations [NIR, Norse] for sufficiently small timesteps. This discrete update rule is widely adopted in SNN simulations because it provides a simple and computationally efficient implementation of membrane dynamics while preserving the exponential decay of the membrane potential through the factor β. For consistency with common practice in the literature, all subsequent simulations in this work use the Standard SNN formulation described above.

Within a fixed discretisation scheme, the discrete simulation is governed by its temporal resolution dt, which interacts with the neuron in two ways. Firstly, it dictates the length of the temporal bin over which the neuron integrates incoming signals. Secondly, it determines rate of decay of the postsynaptic membrane potential via the membrane decay constant β. A higher dt causes a larger decay per timestep, while a smaller dt proportionally decreases the decay per timestep (Supplementary). Fig. 1B illustrates the response of a neuron receiving input of identical mean rate but sampled at different dts. The top panel shows the postsynaptic neuronal dynamics when the input Poisson spike trains are sampled at $dt_1 = 1$ms. The temporal resolution is then increased to $dt_2 = 0.1$ms and the model parameters are scaled to ensure that the magnitude of postsynaptic response is invariant to dt (Appendix 1E, Supplementary). The input is handled in two distinct ways to account for this change: In the first case, the input spike trains are padded with zeros such that the spikes occur at the same physical time (ms) (Fig. 1B, middle). The neuronal dynamics between presynaptic spikes are fully deterministic, and the output spiking activity remains dt-invariant. In the second case, the input spike trains are *resampled* at resolution dt_2 (Fig. 1B, bottom). As a result, the postsynaptic membrane potential now receives near-continuous synaptic input, very little opportunity to decay between incoming updates, and remains closer to threshold longer, firing much more frequently than when the input is padded. This is because when spikes are generated at a fixed discretistion dt, the minimum possible inter-event interval is also dt (Fig. 1C). Resampling the input at a finer resolution therefore allows spikes to occur closer together in time, and the hard minimum inter-event interval on the aggregate input becomes increasingly pronounced as dt decreases.

We now quantify how these effects scale with input firing rate and population size. Fig. 1D compares output spiking rate $r(t)$ of two postsynaptic neurons receiving identical numbers of presynaptic spike trains at the same mean firing rate, but sampled at $dt_1 = 1$ms and $dt_2 = 0.1$ms. The left panel shows the difference in output firing rates as a function of presynaptic population size N and input rate λ, while the right panels report the corresponding Fano factors of the input spike trains at each discretisation. For a fixed number of input neurons, output firing rates diverge as the input rate increases; and for a fixed input rate, divergence increases with the number of presynaptic inputs. When N and λ are both small, both discretisations operate in a dilute regime as $Nrdt << 1$ and its effects remain negligible- the discrete-time spike generation approximates a continuous-time Poisson process. Therefore, two distinct mechanisms underlie discretisation-induced discrepancies:

Mechanism 1- Single neuron breakdown (high input rate): At high λ, coarse discretisation becomes Bernoulli-like, while finer discretisation remains Poisson-like. Inter-spike interval distributions differ at the level of individual neurons, producing divergent postsynaptic responses.

Mechanism 2- Population-level densification of spikes (large N): Each presynaptic neuron satisfies $rdt << n1$ and is individually a Poisson process. However, the superposition of many such processes behaves differently across discretisations: smaller dt leads to denser population-level input, reduced effective decay, and higher postsynaptic firing rates.

Together, these results reveal that matching mean input firing rates across temporal discretisations without preserving spike timing is insufficient to ensure equivalent postsynaptic drive; resampling the input alters the underlying stochastic process. In typical SNN simulations, where spike trains are regenerated at each resolution, re-approximating a continuous-

time system at different dt values must then induce non-trivial changes in spiking output.

2.2 Learning In Feedforward SNNs Degrades as $dt \to 0$

We now extend our analysis from single-neuron to network simulations. In order to systematically evaluate how temporal discretisation impacts learning and generalisation in SNNs, we sought to repeatedly train the network on the same problem while changing the time-step parameters. To this end, we trained networks of spiking LIF neurons (Methods) with a single hidden layer (HL) on a rate-coded MNIST dataset, a standard benchmark to evaluate SNN performance []. Each pixel of a hand-written image was represented as a Poisson spike train and the intensity of each pixel defines the rate of Poisson events, and each image was presented for 100ms. The network then uses a softmax of output membrane potentials to calculate a cross-entropy loss at every time-step to determine the correct digit. Surrogate gradient descent and backpropagation through time (BPTT) algorithms are then used to learn the correct classification (see Methods).

We first consider the scenario where the network is trained on input spike trains generated at a series of dts spanning 0.1ms-1ms. The network is trained and tested at the same dt in each condition. All other network parameters remain unchanged, and the mean input firing rate is held at 50Hz in all cases. Despite matching training and testing discretisations, we observe performance degradation as $dt \to 0$ (Fig. 2A). The origin of this discrepancy lies in the mechanism observed in Fig 1. As the resolution becomes finer, the minimum ISI decreases, and the large presynaptic population causes some hidden layer neurons to receive near-continuous input and produce higher firing rates than at coarse dts (Fig. 2B). This results in the downstream output spiking statistics get altered in the forward pass (Fig. 2C). Since the loss is highly sensitive to the spike counts at every time-step, these perturbations result in a shift in backpropagation paths. Consequently, the variance of the gradients become larger and the loss landscape becomes rougher (Fig. 2D-E), making updates less predictable and training harder as $dt \to 0$.

Overall, although one might expect the training algorithm to converge and better approximate continuous dynamics as the simulation time-step is made smaller, we find that this is not the case. Finer discretisation does not provide any additional independent information; instead, the downstream signals become more redundant and noisy.

2.3 SNNs Learn a dt-Dependent Discrete Dynamical System

Next, we explore the extent to which an SNN is dependent on the chosen discretisation scheme. Toward this end, we study how the network behaves when it is trained and tested on the exact same input data, where the timings of spikes in ms are preserved across the train-test sample, but the temporal resolution of training and testing are different. As a proof-of-principle, we begin by showing the trivial case where the network's train and test dt is set to 1ms. As expected, the loss curves are perfectly matched (Fig. 3A). Next, we upsample the test data samples, where the network is trained at $dt_1 = 1$ms and tested at $dt_2 = 0.1$ms. In other words, the timings of the spikes in ms is matched in both train and test samples, and the test input is padded with zeros to preserve spike times (i.e. no resampling). Despite identical input event times, test performance drops drastically (Fig. 3B). Lastly, we explore the effects of downsampling the test data samples, where train $dt = 0.1$ms and test $dt = 1$ms: the timing of test input events is matched again, but this time it is collapsed into a single time-bin corresponding to the nearest integer in ms, making its resolution 10x lower. We observe that the network overfits to the training data, and the loss curves diverge as training progresses (Fig. 3C).

In effect, the network experiences internal shift: the input is functionally identical, yet the learned mapping no longer produces the correct output. The network has not learned a continuous-time function, but rather a dt-dependent discretised dynamical system. This is not contradictory to the toy single neuron scenario we showed in Fig. 1B, middle. The network now contains learned weights, temporally recurrent interactions, and hidden layer states. During training, the network develops dt_1-specific dynamics: weights encode expected membrane decay, spike density, and timing of threshold crossings. Testing at dt_2 produces completely different voltage trajectories. Small differences amplify downstream and produce diverging spike times and counts in the hidden and output layers, leading to large deviations in loss. These results show that SG trained SNNs heavily exploit discrete statistics but do not survive changes in discretisation methods, preventing generalisation between discrete systems.

2.4 Discretisation Effects Are Overcome Using High Input Rates, Deeper Networks and Longer Membrane Constants

A natural question might arise at this point- if the effects of discretisation are so stark, how come decades of studying SNNs have not brought them forth? The answer is that these effects only become apparent in simpler (and arguably more realistic) regimes. Most theoretical studies mask these effects in one of the following ways:

1. High input rates: Input rates range anywhere from 100-1000Hz in conventional SNN studies. When input rates are high, the network operates in a near-continuous regime; changes in dt are not stark and performance begins converging beyond a certain rate (Fig 4A).

2. Deeper networks: As the number of hidden layers increase, multiple combinations of hidden layer neurons can account for the same target, diversifying the credit assignment per class. This results in a dt-invariant system; the erratic spiking outliers induced by making resolution finer do not have as much control over the network anymore and drown (Fig. 4B).

3. Longer membrane constants: increasing the membrane time constant τ_m of the hidden and output layer neurons allows longer decay per time-step even at lower dts, preventing the membrane potential from hovering near the threshold and contributing to unaccounted spikes (Fig. 4C).

2.5 Calculating Loss On The Average Membrane Potential Can Recover Performance In Dilute Regimes

We go back to our original SNN operating in a dilute regime: an SNN with a single 100-neuron hidden layer being trained on input data with a mean rate of 50 Hz. Until now, the loss per trial L was a 'per-step loss' calculated as

$$L = \sum_{k=0}^{N-1} \ell(V[k]) \tag{2}$$

where,

$$\ell(V[k]) = -\sum_{i=0}^{C} y_i \log s(V_i[k]) \tag{3}$$

is the per-step cross entropy loss calculated at every time-step k for N time-steps between the one-hot target vector y and the softmax s of the ith neuron's membrane potential V_i

$$s(V_i[k]) = \frac{e^{V_i[k]}}{\sum_{i=0}^{C} e^{V_i[k]}}$$

The gradient of the loss with respect to the membrane potential (the local error signal) at time-step k is then:

$$b[k] := \frac{\partial L}{\partial V[k]} = \frac{\partial \ell[k]}{\partial V[k]} = s[k] - y \tag{4}$$

This has profound consequences on both the forward and backward passes of the network. An error term is injected at every k, so the backward dynamics are being constantly forced by per-step supervision. The gradient encourages fine-grained time-local corrections. This encourages the network to adjust $V[k]$ every dt ms and be correct instantaneously. As a result, even slight membrane advantage is enough, and the membrane potential often hovers near threshold during the forward pass. As we saw in Fig 1-2, a lower dt amplifies these sensitive, near-threshold spike effects, leading to a decay in performance as $dt \to 0$. Let us now redefine the loss into an 'average loss' as follows:

$$L = \ell(\frac{1}{N}\sum_{k=0}^{N-1} V[k]) \tag{5}$$

Further, let's define

$$\bar{V} = \frac{1}{N}\sum_{k=0}^{N-1} V[k] \tag{6}$$

Then,

$$b[k] := \frac{\partial L}{\partial V[k]} = \frac{1}{N}\frac{\partial L}{\partial \bar{V}} \tag{7}$$

So:

$$b[k] = \frac{1}{N}(s(\bar{V}) - y) \tag{8}$$

The error is now a coherent quantity which remains uniform across time-steps and depends on the global average. It is also scaled by $\frac{1}{N} := \frac{dt}{T}$, where T is the simulation duration. In this case, the network is encouraged to create sustained class dominance, increase overall separation, and not rely on marginal per-step wins. The dynamics across neurons are pushed away from near threshold jitter and frequent marginal crossings. Under the average loss

regime, the network's performance improves significantly across resolutions for both a 1-layer and 2-layer, and the disparity between learning at different dts is also minimised (Fig 5 A-C). The average loss regime also improves performance in the scenario where testing data is upsampled (downsampled) training data (Fig. 5D). We then decrease the input rates to realistic cortical stimulation activity of 5 Hz, and show that the average loss also improves the network's performance in this dilute regime, unlike the per-step loss (Fig. 5E). The complete derivation of all quantities in this section can be found in Appendix 2.

2.6 Recurrent Spiking Neural Network

Up to here, we analysed the dependence of performance of SG-trained SNNs on discretisation in feed-forward networks. We now investigate how training depends on discretisation and loss definitions in recurrently connected networks of spiking neurons, or recurrent spiking neural networks (RSNNs).
We introduce recurrence in the hidden layer at every time-step k through the current that comes from the previous time-step's hidden spikes $S[k-1]$, via the weight matrix W_{rec}. In other words:

$$I_{rec} = W_{rec} S[k-1] \tag{9}$$

We find that RSNNs also show dt-dependent discretisation effects (Fig. 6A) which can be overcome by modifying the per-step loss into an average loss (Fig. 6B).

—Methods + Discussion to be written—

Appendix

1. Discretisation and Rescaling Methods

In this section, we describe the four discretisation methods shown in Fig. 1A from the continuous LIF equation. For each discretisation, we also derive the relevant input rescaling factor which ensures that the output voltage response and spike timings remain preserved across dts. This rescaling compensates for the fact that when the simulation time-step changes, the number of discrete integration steps per unit time changes. Without this adjustment, the total input accumulated by the neuron over a fixed physical time interval would scale with dt, altering the neuron's response even when the underlying input signal remains unchanged. Specifically, we express the synaptic weight at an arbitrary time-step dt' in terms of a reference weight $dt = dt_{ref}$.

1A. The LIF Neuron

The continuous-time sub-threshold membrane dynamics of an LIF neuron can be modelled as:

$$\tau_m \frac{dV(t)}{dt} = -V(t) + RI(t) \tag{10}$$

where $I(t)$ is the input driving current and $\tau_m = RC$ is the membrane time constant. The neuron is a leaky integrator where the membrane potential decays towards rest unless driven by input. Further, the input current $I(t)$ can be written as:

$$I(t) = WX(t) \tag{11}$$

where inputs $X(t)$ are scaled by learnable afferent weights W.
More generally, the dynamics in Eq. (10) can be expressed as:

$$\dot{V}(t) = AV(t) + Bu(t) \tag{12}$$

where $A = -1/\tau_m$ and $u(t) = I(t) = WX(t)$ is the input signal scaled by $B = 1/\tau_m$. Here, we have assumed $R = 1$ without loss of generality.
Next, we define a discrete-time index k such that $kdt \leq t < (k+1)dt$, where dt is the sampling interval. Then, the continuous input $u(t)$ can be given in terms of the discrete input $u[k]$ as:

$$u(t) = u[k] = WX[k] \tag{13}$$

The discrete samples of the membrane potential can also be written as:

$$V[k] = V(kdt) \tag{14}$$

The goal of discretisation is to find a relation between $u[k]$ and $V[k]$ such that:

$$V[k+1] = \bar{a}V[k] + \bar{b}u[k+1] \tag{15}$$

where $\bar{a}$ and $\bar{b}$ are the discrete-time equivalents of continuous parameters A and B. The exact form of $\bar{a}$ and $\bar{b}$ depends on the chosen discretisation strategy, resulting in each method approximating the neuronal dynamics differently.
We also introduce the spiking and reset mechanisms of the LIF neuron, where if the membrane exceeds the threshold, then the neuron emits an output spike:

$$S[k] = \begin{cases} 1, & \text{if } V[k] \geq \theta \\ 0, & \text{otherwise} \end{cases} \tag{16}$$

If the neuron emits a spike ($S[k] = 1$), the membrane potential is reset to 0; otherwise, the reset term has no effect ($S[k] = 0$):

$$V[k] = (V[k]_{pre-reset})(1 - S[k]) \tag{17}$$

1B. Euler's Forward Discretisation

Euler's forward discretisation provides a simple finite-difference approximation, where the sub-threshold membrane potential of a single LIF neuron described in Eq. (10) can be written as:

$$V[k+1] = V[k] + \frac{dt}{\tau_m}(-V[k] + WX[k+1]) \tag{18}$$

or,

$$V[k+1] = (1 - \frac{dt}{\tau_m})V[k] + \frac{dt}{\tau_m}u[k+1] \tag{19}$$

Defining $\alpha = 1 - dt/\tau_m$, we can write:

$$V[k+1] = \alpha V[k] + (1-\alpha)u[k+1] \tag{20}$$

Hence, from Eq. 15, $\bar{a} = \alpha$ and $\bar{b} = 1-\alpha$. Spiking and reset dynamics follow Eq. (17). Input magnitudes must therefore be rescaled by a factor of dt_{ref}/dt' so that the total input integrated over one unit of physical time remains invariant when the simulation time-step changes. The Euler method can be sufficient for deterministic or slowly varying inputs when $dt \ll \tau_m$.

1C. Zero-Order Hold Discretisation

For more "exact" discretisation, we write the general solution of Eq. (10) as:

$$V(t+dt) = e^{Adt}V(t) + \int_0^{dt} e^{A(dt-\eta)} B\, u(t+\eta)\, d\eta \quad (21)$$

In zero-order hold (ZOH) discretisation, the input is assumed to remain constant over each time-step k of duration dt, allowing the discretised response to exactly match the continuous-time response to a step input, so that

$$u(t+\eta) = u[k+1], \quad 0 \le \eta < dt$$

Substituting this into Eq. (21), we get the ZOH membrane potential:

$$V[k+1] = e^{Adt}V[k] + (\int_0^{dt} e^{A(dt-\eta)} B\, d\eta) u[k+1] \quad (22)$$

From Eq. 15, this gives us $\bar{a} = e^{Adt}$ and $\bar{b} = \int_0^{dt} e^{A(dt-\eta)} B\, d\eta$.

Now,

$$\int_0^{dt} e^{A(dt-\eta)} d\eta = \int_0^{dt} e^{-(dt-\eta)/\tau_m} d\eta \quad (23)$$

$$= \tau_m (1 - e^{-dt/\tau_m}) \quad (24)$$

since $A = -1/\tau_m$. The effective input gain for a time-step dt therefore becomes

$$\bar{b}(dt) = 1 - e^{-dt/\tau_m}$$

Substituting the expressions for $\bar{a}$ and $\bar{b}$ yields the exact discrete-time LIF dynamics under a zero-order hold assumption:

$$V[k+1] = e^{-dt/\tau_m} V[k] + (1 - e^{-dt/\tau_m}) u[k+1] \quad (25)$$

If the synaptic weight is defined relative to a reference time-step dt_{ref}, the effective weight must scale proportionally to $\bar{b}(dt)$. Hence, weight at a reference time-step dt_{ref} can then be expressed as:

$$W(dt_{ref}) = (1 - e^{-dt_{ref}/\tau_m}) \quad (26)$$

Our goal is to obtain a closed-form relationship for $W(dt)$ such that scaling the weights compensates for changes in dt. For this, using Eq. (24), we can derive the scaling factor:

$$\frac{W(dt')}{W(dt_{ref})} = \frac{1 - e^{-dt'/\tau_m}}{1 - e^{-dt_{ref}/\tau_m}} \quad (27)$$

1D. Impulse-Invariant Discretisation

In impulse-invariant discretisation (IID), the input is modelled as a sequence of instantaneous impulses. The continuous-time impulse response of the system is therefore preserved in the discrete-time model. The continuous-time impulse response of the system described in Eq. (10) is given by

$$V(t) = \int_0^t e^{A(t-\eta)} B\delta(\eta) d\eta = e^{At} B \quad (28)$$

Discretising this impulse response with $t = kdt$, we obtain:

$$g_{DT}[k] = V[k] = e^{Akdt} B \quad (29)$$

$$= \bar{a}^k B \quad (30)$$

where we define $\bar{a} = e^{Adt}$.

Next, consider the recursive form of Eq. (15) under unit discrete impulse input:

$$V[0] = \bar{b} \quad (31)$$

$$V[1] = \bar{a}\bar{b} \quad (32)$$

$$V[2] = \bar{a}^2\bar{b} \quad (33)$$

$$\text{or, } V[k] = \bar{a}^k\bar{b} \quad (34)$$

for $k \ge 0$. From Eq. 30 and Eq. 34, we obtain

$$\bar{b} = B = 1/\tau_m \quad (35)$$

Substituting the expressions for $\bar{a}$ and $\bar{b}$, the discrete-time LIF dynamics under impulse-invariant discretisation become

$$V[k+1] = e^{-dt/\tau_m} V[k] + \frac{1}{\tau_m} u[k+1] \quad (36)$$

Since the input term is given by $u[k+1] = WX[k+1]$, any dt-dependent input gain $\bar{b}$ can equivalently be absorbed into the effective synaptic weight multiplying $X[k+1]$. Under impulse-invariant discretisation, however, we obtained $\bar{b} = B$, which is independent of dt. Therefore the effective synaptic weight does not require any rescaling across time-steps, and we get:

$$\frac{W(dt')}{W(dt_{ref})} = 1 \quad (37)$$

1E. Standard SNN Discretisation

In many spiking neural network implementations, the membrane dynamics are written directly in discrete time rather than derived from a specific continuous-time discretisation scheme. A commonly used update rule is

$$V[k+1] = \beta V[k] + WX[k+1] \tag{38}$$

where $\beta = e^{-dt/\tau_m}$ represents the membrane decay constant for a timestep dt, and $WX[k+1]$ represents the weighted input arriving at the neuron at timestep $(k+1)$.
Comparing Eq. (38) with the general discrete form introduced in Eq. (15), we can identify

$$\bar{a} = e^{-dt/\tau_m}, \qquad \bar{b} = 1 \tag{39}$$

Unlike the discretisation schemes derived previously, the input gain $\bar{b}$ is fixed to unity and does not depend on the timestep dt. Since the input is given by $u[k+1] = WX[k+1]$, the effective scaling of synaptic input is therefore determined solely by the weight W.
Consequently, when the timestep changes from a reference value dt_{ref} to a new timestep dt', maintaining the same effective input magnitude requires

$$\frac{W(dt')}{W(dt_{ref})} = 1 \tag{40}$$

Thus, under the standard SNN formulation, synaptic weights are typically kept unchanged when the timestep varies. In contrast to the Euler and ZOH discretisations, where the input gain explicitly depends on dt, this formulation assumes that the magnitude of synaptic input is independent of the simulation timestep.

2. SNN Gradient Derivation

In this section, we derive the exact equations of the forward pass and then derive the backward pass and parameter gradients for the output layer of a spiking network composed of LIF neurons within the surrogate-gradient approximation. The neuron model remains the same as the one described in Methods.

We start by deriving the form of these equations for a general discrete-time dynamical system, and then re-writing them for our system of LIF neurons.

2A. Backpropagation Through Time via Lagrange Multipliers

Consider a general discrete-time dynamical system

$$V[k] = f_k(V[k-1], W), \qquad k = 0, \dots, N-1 \tag{41}$$

with total loss

$$L(W) = \sum_{k=0}^{N-1} \ell(V[k]). \tag{42}$$

The forward dynamics impose equality constraints between successive states. To compute gradients efficiently, we introduce Lagrange multipliers $p_k \in \mathbb{R}^n$ enforcing these constraints.
The Lagrangian of the constrained optimisation problem is:

$$\mathcal{L} = \sum_{k=0}^{N-1} \ell(V[k]) + \sum_{k=0}^{N-1} p_k^\top \big(f_k(V[k-1], W) - V[k]\big) \tag{43}$$

where p_k are the adjoint variables (also called co-states).

An optimum must satisfy the stationarity conditions of the Lagrangian with respect to all variables.

Forward dynamics
Taking derivatives with respect to the multipliers gives:

$$\frac{\partial \mathcal{L}}{\partial p_k} = 0 \quad \Rightarrow \quad V[k] = f_k(V[k-1], W) \tag{44}$$

which simply recovers the forward system dynamics.

Co-state equations
Taking derivatives of the Lagrangian with respect to the state $V[k]$ for $k = 0, \dots, N-2$ and setting $\frac{\partial \mathcal{L}}{\partial V[k]} = 0$ gives:

$$0 = \frac{\partial \ell(V[k])}{\partial V[k]} - p_k + \left(\frac{\partial f_{k+1}}{\partial V[k]}\right)^\top p_{k+1} \tag{45}$$

Rearranging yields the backward recursion:

$$p_k = \left(\frac{\partial f_{k+1}}{\partial V[k]}\right)^\top p_{k+1} + \frac{\partial \ell[k]}{\partial V[k]} \tag{46}$$

The terminal condition follows from the derivative with respect to the final state $V[N-1]$

$$p_{N-1} = \frac{\partial \ell[N-1]}{\partial V[N-1]} \tag{47}$$

Thus, the adjoint variables p_k satisfy the gradient of the total loss with respect to the state $V[k]$

$$p_k = \frac{\partial L}{\partial V[k]} \tag{48}$$

Gradient with respect to parameters
Differentiating the Lagrangian with respect to the parameters gives:

$$\nabla_W L = \sum_{k=0}^{N-1} \left(\frac{\partial f_k}{\partial W}\right)^\top p_k. \tag{49}$$

This expression shows that the gradient can be computed once the forward trajectory $V[k]$ and the backward co-states p_k are known.

2B. LIF Neuron Network

For the neuron model in Eq.() the state update is

$$f_k(V[k-1], W) = (1 - S[k]) \odot (\beta V[k-1] + W^\top x[k]). \tag{50}$$

Applying the general results above yields the forward, backward, and gradient expressions derived in the following sections.

Forward Pass
Defining $U[k]$ as the pre-reset potential, we can rewrite the LIF neuron model as:

$$U[k] = \beta V[k-1] + W^\top x[k] \tag{51}$$
$$S[k] = H(U[k] - \theta) \tag{52}$$
$$V[k] = (1 - S[k])\, U[k] \tag{53}$$

where $H(\cdot)$ denotes the Heaviside step function applied elementwise. Therefore, the forward dynamics of the network at timestep k consists of the following steps:

1. $U[k] = \beta V[k-1] + W^\top x[k]$
2. $S[k] = H(U[k] - \theta)$
3. $V[k] = (1 - S[k])U[k]$

For classification, the membrane potentials are mapped to class probabilities via a softmax

$$s[k] = \text{softmax}(V[k]) \tag{54}$$

and the instantaneous loss is defined as the cross-entropy

$$\ell[k] = -y^\top \log s[k] \tag{55}$$

where y is a one-hot target vector.

Local Error Signal Derivative
For the softmax and cross-entropy loss, the local error signal, the derivative of the instantaneous loss with respect to the membrane potential is

$$b_k := \frac{\partial \ell[k]}{\partial V[k]} = s[k] - y \tag{56}$$

Jacobian
The Jacobian of the state update

$$V[k] = (1 - S[k])(\beta V[k-1] + W^\top x[k])$$

can be obtained by applying the chain rule. Since the spiking nonlinearity acts element-wise, the resulting Jacobians are diagonal with respect to the neuron index.
From eq. 21 and eq. 22, we obtain:

$$\frac{\partial V[k]}{\partial U[k]} = (1 - S[k]) - U[k]H'(U[k] - \theta) \tag{57}$$

Using the dependence of $U[k]$ on the previous state and weights then gives:

$$\frac{\partial V[k]}{\partial V[k-1]} = \beta\big((1 - S[k]) - U[k]H'(U[k] - \theta)\big) \tag{58}$$

$$\frac{\partial V[k]}{\partial W} = x[k]\big((1 - S[k]) - U[k]H'(U[k] - \theta)\big)^\top \tag{59}$$

Backward Pass
Let the co-states be

$$p_k := \frac{\partial L}{\partial V[k]}$$

From the adjoint derivation above, the backward recursion is

$$p_k = \left(\frac{\partial V[k+1]}{\partial V[k]}\right)^\top p_{k+1} + b_k \tag{60}$$

with terminal condition

$$p_{N-1} = b_{N-1} \tag{61}$$

Substituting the Jacobian derived in the previous section gives

$$p_k = \beta\Big((1-S[k+1]) - U[k+1]H'(U[k+1]-\theta)\Big) \odot p_{k+1} + b_k \tag{62}$$

Since the derivative of the hard threshold $H'(U - \theta)$ is not usable for gradient-based optimisation in

SNNs, we replace it with a smooth surrogate derivative $\sigma'(U-\theta)$ in the backward pass. This yields

$$p_k = \beta\Big((1-S[k+1])-U[k+1]\sigma'(U[k+1]-\theta)\Big)\odot p_{k+1}+b_k \tag{63}$$

Defining the Jacobian factor

$$J_{k+1} := \beta\big((1-S[k+1])-U[k+1]\sigma'(U[k+1]-\theta)\big) \tag{64}$$

the recurrence becomes

$$p_k = b_k + J_{k+1}\odot p_{k+1} \tag{65}$$

Unrolling the co-states:

$$p_k = b_k + J_{k+1}\odot b_{k+1} + J_{k+1}\odot J_{k+2}\odot b_{k+2} + \cdots$$

which can be written as

$$p_k = \sum_{t=k}^{N-1}\left(\prod_{m=k+1}^{t} J_m\right)\odot b_t \tag{66}$$

where

$$J_m = \beta\big((1-S[m])-U[m]\sigma'(U[m]-\theta)\big)$$

For a fixed simulation time T, the timestep is

$$dt = \frac{T}{N}, \qquad \beta = e^{-dt/\tau} = \exp\left(-\frac{T}{\tau N}\right)$$

Hence

$$\beta^{t-k} = \exp\left(-(t-k)\frac{T}{\tau N}\right) = \exp\left(-\frac{T}{\tau}\frac{t-k}{N}\right) \tag{67}$$

Thus, the product of Jacobians can be written as:

$$\prod_{m=k+1}^{t} J_m = \exp\left(-\frac{T}{\tau}\frac{t-k}{N}\right)\times \prod_{m=k+1}^{t}\Big((1-S[m])-U[m]\sigma'(U[m]-\theta)\Big) \tag{68}$$

Substituting into the unrolled costate gives

$$p_k = \sum_{t=k}^{N-1}\exp\left(-\frac{T}{\tau}\frac{t-k}{N}\right)\times \left(\prod_{m=k+1}^{t}\big((1-S[m])-U[m]\sigma'(U[m]-\theta)\big)\right)\odot b_t \tag{69}$$

Gradient

The total gradient with respect to W is

$$\nabla_W L = \sum_{k=0}^{N-1}\left(\frac{\partial V[k]}{\partial W}\right)^{\top} p_k \tag{70}$$

Using the Jacobian derived earlier and the surrogate derivative σ', we obtain:

$$\frac{\partial V[k]}{\partial W} = x[k]\big((1-S[k])-U[k]\sigma'(U[k]-\theta)\big)^{\top} \tag{71}$$

Substituting this expression gives

$$\nabla_W L = \sum_{k=0}^{N-1} x[k]\Big((1-S[k])-U[k]\sigma'(U[k]-\theta)\Big)^{\top} p_k \tag{72}$$

Substituting Eqs. () into the weight gradient yields:

$$\nabla_W L = \sum_{k=0}^{N-1} x[k]\Big((1-S[k])-U[k]\sigma'(U[k]-\theta)\Big)^{\top}\times \left[\sum_{t=k}^{N-1} e^{-\frac{T}{\tau}\frac{t-k}{N}}\left(\prod_{m=k+1}^{t}\big((1-S[m]) - U[m]\sigma'(U[m]-\theta)\big)\odot b_t\right]\right. \tag{73}$$

2C. Average Loss Objective

So far, we considered a per-timestep loss of the form

$$L = \sum_{k=0}^{N-1}\ell[k], \qquad \ell[k] = \ell(V[k])$$

We now consider instead a loss applied to the time-averaged membrane potential:

$$L = \ell(\bar{V}), \qquad \bar{V} := \frac{1}{N}\sum_{k=0}^{N-1} V[k] \tag{74}$$

Thus, unlike the previous case, there is no longer a separate instantaneous loss at each timestep. Instead, a single loss is evaluated after first averaging the membrane potentials across time.

For classification, we define

$$s = \mathrm{softmax}(\bar{V}), \qquad L = -y^{\top}\log s \tag{75}$$

where y is a one-hot target vector.

Derivative with respect to the state

Using the chain rule,

$$\frac{\partial L}{\partial V[k]} = \frac{\partial L}{\partial \bar{V}} \frac{\partial \bar{V}}{\partial V[k]} \tag{76}$$

Since

$$\bar{V} = \frac{1}{N} \sum_{k=0}^{N-1} V[k]$$

we have

$$\frac{\partial \bar{V}}{\partial V[k]} = \frac{1}{N} I \tag{77}$$

where I is the identity matrix. Therefore,

$$b_k := \frac{\partial L}{\partial V[k]} = \frac{1}{N} \frac{\partial L}{\partial \bar{V}} \tag{78}$$

For softmax and cross-entropy loss,

$$\frac{\partial L}{\partial \bar{V}} = s - y \tag{79}$$

so that

$$b_k = \frac{1}{N}(s - y), \qquad k = 0, \ldots, N-1 \tag{80}$$

Hence, in the average-loss setting, the direct error signal injected at each timestep is identical across time and scaled by $1/N$.

Backward recursion

The co-state recursion retains the same form as before,

$$p_k = \left(\frac{\partial V[k+1]}{\partial V[k]} \right)^\top p_{k+1} + b_k \tag{81}$$

with terminal condition

$$p_{N-1} = b_{N-1} \tag{82}$$

Under the surrogate-gradient approximation, this becomes

$$p_k = J_{k+1} \odot p_{k+1} + \frac{1}{N}(s - y) \tag{83}$$

where

$$J_{k+1} := \beta\big((1 - S[k+1]) - U[k+1]\sigma'(U[k+1] - \theta)\big) \tag{84}$$

Unrolling the recursion gives

$$p_k = \sum_{t=k}^{N-1} \left(\prod_{m=k+1}^{t} J_m \right) \odot \frac{1}{N}(s - y) \tag{85}$$

Equivalently,

$$p_k = \frac{1}{N} \sum_{t=k}^{N-1} \left(\prod_{m=k+1}^{t} J_m \right) \odot (s - y) \tag{86}$$

Gradient with respect to W

The gradient with respect to the weights is still

$$\nabla_W L = \sum_{k=0}^{N-1} \left(\frac{\partial V[k]}{\partial W} \right)^\top p_k \tag{87}$$

Using the surrogate Jacobian,

$$\frac{\partial V[k]}{\partial W} = x[k]\big((1 - S[k]) - U[k]\sigma'(U[k] - \theta)\big)^\top \tag{88}$$

we obtain

$$\nabla_W L = \sum_{k=0}^{N-1} x[k]\big((1 - S[k]) - U[k]\sigma'(U[k] - \theta)\big)^\top p_k \tag{89}$$

Substituting the unrolled co-states yields

$$\nabla_W L = \frac{1}{N} \sum_{k=0}^{N-1} x[k]\big((1 - S[k]) - U[k]\sigma'(U[k] - \theta)\big)^\top \times \left[\sum_{t=k}^{N-1} \left(\prod_{m=k+1}^{t} J_m \right) \odot (s - y) \right] \tag{90}$$

Thus, relative to the per-timestep loss, the average-loss objective changes the gradient in two ways: first, the direct error signal is shared uniformly across all timesteps; second, this signal is scaled by a factor of $1/N$ before being propagated backward through the temporal Jacobians.